Lab 01 Go Foundation - Hands-on Learning

Duration: 2.5 hours | **Points:** 100

Lab Overview

Objective: Learn Go fundamentals by building a **Simple Property Calculator** step by step. You'll start with provided code examples and extend them to create your own functions.

What You'll Build: A property analysis tool that calculates prices, validates input, and provides investment recommendations.

Learning Style:

- 📖 **Read** sample code
- 🧐 **Understand** the concept
- ✍️ **Write** your own code based on examples
- 🚀 **Extend** with new features

Setup (15 minutes)

Step 1: Install Go

Download from https://golang.org/dl/ and verify:

```
go version  # Should show go1.21.x or higher
```

Step 2: Create Project

```
mkdir go-property-calculator
cd go-property-calculator
go mod init property-calculator
```

Step 3: VS Code Setup

Install **Go extension** by Google

Part 1: Variables and Basic Types (25 minutes)

📖 Sample Code: Basic Property Data

Create `main.go` with this starter code:

```go
package main

import "fmt"

func main() {
    // Sample property data
    var propertyName string = "Saigon Apartment"
    var price float64 = 2500000000  // 2.5 billion VND
    var area float64 = 75.5         // 75.5 square meters
    var bedrooms int = 2
    var isAvailable bool = true

    // Calculate price per square meter
    pricePerM2 := price / area

    fmt.Println("=== Property Information ===")
    fmt.Printf("Name: %s\n", propertyName)
    fmt.Printf("Price: %.0f VND\n", price)
    fmt.Printf("Area: %.1f m²\n", area)
    fmt.Printf("Bedrooms: %d\n", bedrooms)
    fmt.Printf("Available: %t\n", isAvailable)
    fmt.Printf("Price per m²: %.0f VND\n", pricePerM2)
}
```

Test: Run `go run main.go` and verify it works.

✍️ Your Task 1.1: Add More Properties (10 min)

Challenge: Extend the code to handle 3 properties and find the cheapest one.

What to add:

1. Create variables for 2 more properties (property2, property3)
2. Calculate price per m^2 for all 3
3. Find and print which property has the lowest price per m^2

Hints:

```go
// Property 2 data
var property2Name string = "Your choice"
var property2Price float64 = // Your price
// ... continue with other fields

// Finding minimum (example structure)
cheapestPrice := property1PricePerM2
cheapestName := property1Name

if property2PricePerM2 < cheapestPrice {
    // Update cheapest
```

```
}
// Continue for property3...
```

Expected Output:

```
=== Property Comparison ===
Property 1: Saigon Apartment - 33,113 VND/m²
Property 2: [Your property] - [Your calculation]
Property 3: [Your property] - [Your calculation]

Cheapest per m²: [Property name] at [price] VND/m²
```

✍️ Your Task 1.2: Price Categories (15 min)

Sample Code: Here's how to categorize one property:

```
// Add this after your property data
func categorizeProperty(pricePerM2 float64) string {
    if pricePerM2 > 50000000 {
        return "LUXURY"
    } else if pricePerM2 > 30000000 {
        return "PREMIUM"
    } else if pricePerM2 > 20000000 {
        return "STANDARD"
    }
    return "BUDGET"
}
```

Your Challenge:

1. Use this function to categorize ALL your properties
2. Count how many properties are in each category
3. Create a new function `formatPrice()` that converts VND to billions/millions
Example: `2500000000` → `"2.5 tỷ VND"` - Example: `850000000` → `"850 triệu VND"`

Expected Output:

```
=== Property Categories ===
Saigon Apartment: STANDARD (2.5 tỷ VND)
[Property 2]: [Category] ([Formatted price])
[Property 3]: [Category] ([Formatted price])

Category Summary:
LUXURY: 0 properties
PREMIUM: 1 properties
STANDARD: 2 properties
BUDGET: 0 properties
```

Part 2: Arrays, Slices, and Maps (30 minutes)

📖 Sample Code: Property Collections

Replace your `main.go` with this improved version:

```go
package main

import "fmt"

// Property struct to organize data better
type Property struct {
    Name     string
    Price    float64
    Area     float64
    Bedrooms int
    District string
}

func main() {
    // Array of properties
    properties := []Property{
        {"Saigon Apartment", 2500000000, 75.5, 2, "District 1"},
        {"HCMC House", 4200000000, 120.0, 3, "District 7"},
        {"Budget Studio", 800000000, 35.0, 1, "Binh Thanh"},
    }

    fmt.Println("=== All Properties ===")
    for i, prop := range properties {
        pricePerM2 := prop.Price / prop.Area
        fmt.Printf("%d. %s: %.0f VND (%.0f VND/m²)\n",
            i+1, prop.Name, prop.Price, pricePerM2)
    }
}
```

✍️ Your Task 2.1: Property Search (10 min)

Challenge: Add search functionality.

What to implement:

1. Function `findPropertiesInBudget(properties []Property, maxBudget float64)`
2. Function `findPropertiesByBedrooms(properties []Property, bedrooms int)`
3. Test both functions with your data

Code Structure:

```
func findPropertiesInBudget(properties []Property, maxBudget float64)
[]Property {
    var result []Property

    for _, prop := range properties {
        if /* YOUR CONDITION */ {
            result = append(result, prop)
        }
    }

    return result
}

// In main(), test your functions:
budget := 3000000000.0  // 3 billion VND
affordable := findPropertiesInBudget(properties, budget)
fmt.Printf("\nProperties under %.0f VND:\n", budget)
// Print the results...
```

✍️ Your Task 2.2: District Analysis with Maps (20 min)

Sample Code: Here's how to use maps for district data:

```
// Add this to your imports
import (
    "fmt"
    "sort"
)

// Add this function
func analyzeByDistrict(properties []Property) map[string][]Property {
    districtMap := make(map[string][]Property)

    for _, prop := range properties {
        districtMap[prop.District] = append(districtMap[prop.District], prop)
    }

    return districtMap
}
```

Your Challenge:

1. Use the `analyzeByDistrict` function
2. Create `calculateDistrictStats()` function that returns: - Average price per district - Property count per district - Most expensive property in each district
3. Display results sorted by average price (highest first)

Expected Output:

```
=== District Analysis ===
District 1: 1 properties, Avg: 2.5 tỷ VND, Most expensive: Saigon Apartment
District 7: 1 properties, Avg: 4.2 tỷ VND, Most expensive: HCMC House
Binh Thanh: 1 properties, Avg: 800 triệu VND, Most expensive: Budget Studio

Ranking by Average Price:
1. District 7: 4.2 tỷ VND
2. District 1: 2.5 tỷ VND
3. Binh Thanh: 800 triệu VND
```

Part 3: Functions and Methods (35 minutes)

📖 Sample Code: Property Methods

Add methods to your Property struct:

```go
// Add these methods to Property struct
func (p Property) PricePerM2() float64 {
    if p.Area == 0 {
        return 0
    }
    return p.Price / p.Area
}

func (p Property) IsAffordable(budget float64) bool {
    return p.Price <= budget
}

// Usage example in main():
for _, prop := range properties {
    fmt.Printf("%s: %.0f VND/m² (Affordable with 3B budget: %t)\n",
        prop.Name, prop.PricePerM2(), prop.IsAffordable(3000000000))
}
```

✍️ Your Task 3.1: Investment Calculator (15 min)

Challenge: Create an investment analysis system.

What to implement:

1. Method `(p Property) CalculateROI(monthlyRent float64) float64` - Return annual ROI percentage - Formula: `(monthlyRent * 12 / property.Price) * 100`
2. Method `(p Property) InvestmentGrade() string`
    - Return “EXCELLENT” (ROI > 8%), “GOOD” (5-8%), “FAIR” (3-5%), “POOR” (<3%)

3. Function `findBestInvestment(properties []Property, rents []float64) Property`

Sample rental data:

```
// Monthly rents for each property (same order as properties)
monthlyRents := []float64{25000000, 35000000, 12000000}  // VND per month
```

Expected Output:

```
=== Investment Analysis ===
Saigon Apartment: ROI 12.0% per year - EXCELLENT
HCMC House: ROI 10.0% per year - EXCELLENT
Budget Studio: ROI 18.0% per year - EXCELLENT

Best Investment: Budget Studio (18.0% ROI)
```

✍️ Your Task 3.2: Loan Calculator (20 min)

Sample Code: Basic loan calculation structure:

```
func calculateMonthlyPayment(loanAmount, annualRate float64, years int)
float64 {
    monthlyRate := annualRate / 100 / 12
    numPayments := float64(years * 12)

    if annualRate == 0 {
        return loanAmount / numPayments
    }

    // Monthly payment formula
    return loanAmount * monthlyRate * math.Pow(1+monthlyRate, numPayments) /
           (math.Pow(1+monthlyRate, numPayments) - 1)
}
```

Your Challenge:

1. Add `import "math"` at the top
2. Create `LoanInfo` struct with fields: `LoanAmount`, `MonthlyPayment`, `TotalInterest`
3. Create method `(p Property) CalculateLoan(downPaymentPercent, interestRate float64, years int) LoanInfo` 4. Test with: 20% down payment, 8.5% interest, 20 years

Expected Output:

```
=== Loan Analysis ===
Saigon Apartment:
  Loan Amount: 2.0 tỷ VND (80% of price)
  Monthly Payment: 17.3 triệu VND
  Total Interest: 2.15 tỷ VND over 20 years

HCMC House:
  Loan Amount: 3.36 tỷ VND (80% of price)
  Monthly Payment: 29.1 triệu VND
  Total Interest: 3.62 tỷ VND over 20 years
```

Part 4: Control Flow and Logic (25 minutes)

📖 Sample Code: Property Recommendation Engine

```go
func recommendProperty(p Property, budget, maxMonthlyPayment float64) string
{
    if !p.IsAffordable(budget) {
        return "❌ SKIP - Over budget"
    }

    // Calculate loan (assume 20% down, 8.5% rate, 20 years)
    loanInfo := p.CalculateLoan(20, 8.5, 20)

    if loanInfo.MonthlyPayment > maxMonthlyPayment {
        return "⚠️ CONSIDER - High monthly payment"
    }

    roi := p.CalculateROI(p.Price * 0.012) // Assume 1.2% monthly rent

    if roi > 10 {
        return "🔥 BUY NOW - Excellent ROI"
    } else if roi > 6 {
        return "☑️ GOOD BUY - Solid investment"
    }

    return "🤔 MAYBE - Average investment"
}
```

✍️ Your Task 4.1: Smart Recommendation System (15 min)

Challenge: Enhance the recommendation with more criteria.

Extend the `recommendProperty` function to consider:

1. **Location premium:** District 1, 2, 7 get bonus points
2. **Size efficiency:** Properties 50-100m^2 are "optimal size"
3. **Price per m^2 reasonableness:** Warn if >60M VND/m^2

Your logic structure:

```
func smartRecommendProperty(p Property, budget, maxMonthlyPayment float64)
(string, string) {
    // Basic affordability check (use existing logic)

    var bonus []string  // Store bonus reasons
    var warnings []string  // Store warning reasons

    // Check location premium
    if /* district condition */ {
        bonus = append(bonus, "Premium location")
    }

    // Check size efficiency
    if /* size condition */ {
        bonus = append(bonus, "Optimal size")
    }

    // Check price reasonableness
    if /* price per m2 condition */ {
        warnings = append(warnings, "High price per m²")
    }

    // Combine everything into final recommendation
    recommendation := "😐 NEUTRAL"  // default
    // Your logic to determine final recommendation...

    details := fmt.Sprintf("Bonus: %v, Warnings: %v", bonus, warnings)
    return recommendation, details
}
```

✍️ Your Task 4.2: Property Portfolio Optimizer (10 min)

Challenge: Given a budget, find the best combination of properties.

What to implement:

```
func optimizePortfolio(properties []Property, totalBudget float64) []Property
{
    var portfolio []Property
    remainingBudget := totalBudget

    // Sort properties by ROI (descending)
```

```
    // Your sorting logic here...

    // Greedily add properties while budget allows
    for _, prop := range sortedProperties {
        if /* can afford property */ {
            portfolio = append(portfolio, prop)
            remainingBudget -= prop.Price
        }
    }

    return portfolio
}
```

Test with: - Budget: 8 billion VND - Should return the best combination of properties

Expected Output:

```
=== Portfolio Optimization ===
Budget: 8.0 tỷ VND
Selected Properties:
1. Budget Studio: 800 triệu VND (ROI: 18.0%)
2. Saigon Apartment: 2.5 tỷ VND (ROI: 12.0%)
3. HCMC House: 4.2 tỷ VND (ROI: 10.0%)

Total Invested: 7.5 tỷ VND
Remaining Budget: 500 triệu VND
Portfolio Average ROI: 13.3%
```

Part 5: Putting It All Together (10 minutes)

✍️ Final Task: Complete Property Analyzer

Challenge: Create a main menu system that uses all your functions.

Starter code:

```
func main() {
    // Your property data here...

    for {
        fmt.Println("\n=== Property Analyzer Menu ===")
        fmt.Println("1. View all properties")
        fmt.Println("2. Search by budget")
        fmt.Println("3. Investment analysis")
        fmt.Println("4. Loan calculator")
        fmt.Println("5. Get recommendations")
        fmt.Println("6. Optimize portfolio")
        fmt.Println("0. Exit")
```

```
        fmt.Print("Choose option: ")

        var choice int
        fmt.Scanln(&choice)

        switch choice {
        case 1:
            // Call your view function
        case 2:
            // Call your search function
        case 3:
            // Call your investment analysis
        // ... implement other cases
        case 0:
            fmt.Println("Goodbye!")
            return
        default:
            fmt.Println("Invalid option!")
        }
    }
}
```

Submission Requirements (100 points)

Code Implementation (70 points)

- ☐ **Task 1.1:** Property comparison working (10 points)
- ☐ **Task 1.2:** Price categorization and formatting (10 points)
- ☐ **Task 2.1:** Property search functions (10 points)
- ☐ **Task 2.2:** District analysis with maps (15 points)
- ☐ **Task 3.1:** Investment calculator methods (10 points)
- ☐ **Task 3.2:** Loan calculation system (15 points)

Logic and Features (20 points)

- ☐ **Task 4.1:** Smart recommendation system (10 points)
- ☐ **Task 4.2:** Portfolio optimizer (10 points)

Code Quality (10 points)

- ☐ Clean, readable Go code (5 points)
- ☐ Proper error handling (3 points)
- ☐ Good variable names and comments (2 points)

Bonus Points (+15)

- ☐ Menu system working (+5 points)

- ☐ Additional features (user input validation, more analysis) (+5 points)
- ☐ Creative enhancements (+5 points)

Testing Your Code

Before submission, verify:

1. All functions compile and run
2. Property comparison shows correct cheapest option
3. Search functions return expected results
4. Investment calculations are reasonable
5. Loan calculations match expected formulas
6. Recommendation system provides logical advice

Getting Help

During Lab:
- 🤖 **Use AI:** "Help me implement property ROI calculation in Go"
- 👥 **Ask classmates:** Share ideas and debug together
- 👨‍🏫 **Instructor:** For concept clarification

Useful AI Prompts:
- "How do I sort a slice of structs by a field in Go?"
- "Help me debug this Go function that calculates monthly payments"
- "What's the best way to handle user input in Go?"

Great job building your first Go application! 🎉